

AI ASSISTED CODING (ASSIGNMENT-3.1)

NAME: T. Ashwin

BATCH NO: 24

ROLL NO: 2503A52700

Q1. Zero-Shot Prompting (Palindrome Number Program)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a palindrome.

Task:

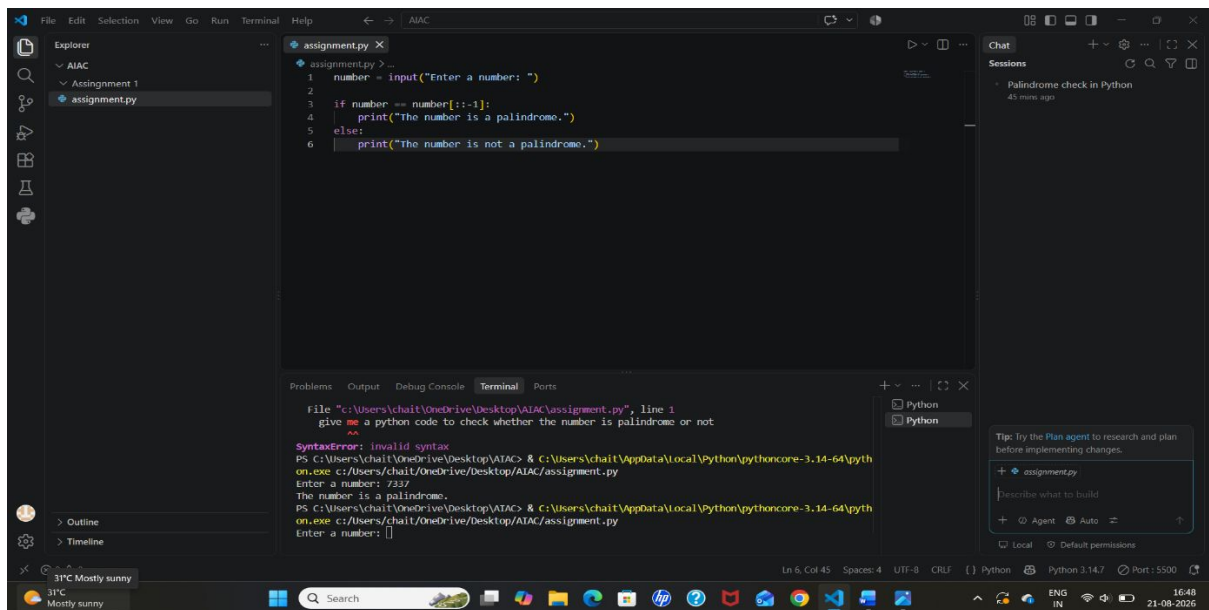
- Record the AI-generated code.
- Test the code with multiple inputs.
- Identify any logical errors or missing edge-case handling.

Prompt: give a python code to check whether the number is palindrome or not

Code:

```
number = input("Enter a number: ")
if number == number[::-1]:
    print("The number is a palindrome.")
else:
    print("The number is not a palindrome.")
```

output:



Q2. One-Shot Prompting (Factorial Calculation)

Write a one-shot prompt by providing one input-output example and ask the AI to generate a Python function to compute the factorial of a given number.

Example:

Input: 5 → Output: 120

Task:

- Compare the generated code with a zero-shot solution.
- Examine improvements in clarity and correctness.

Prompt: generate a python code to calculate the factorial of a given number

Code:

```
number = int(input())

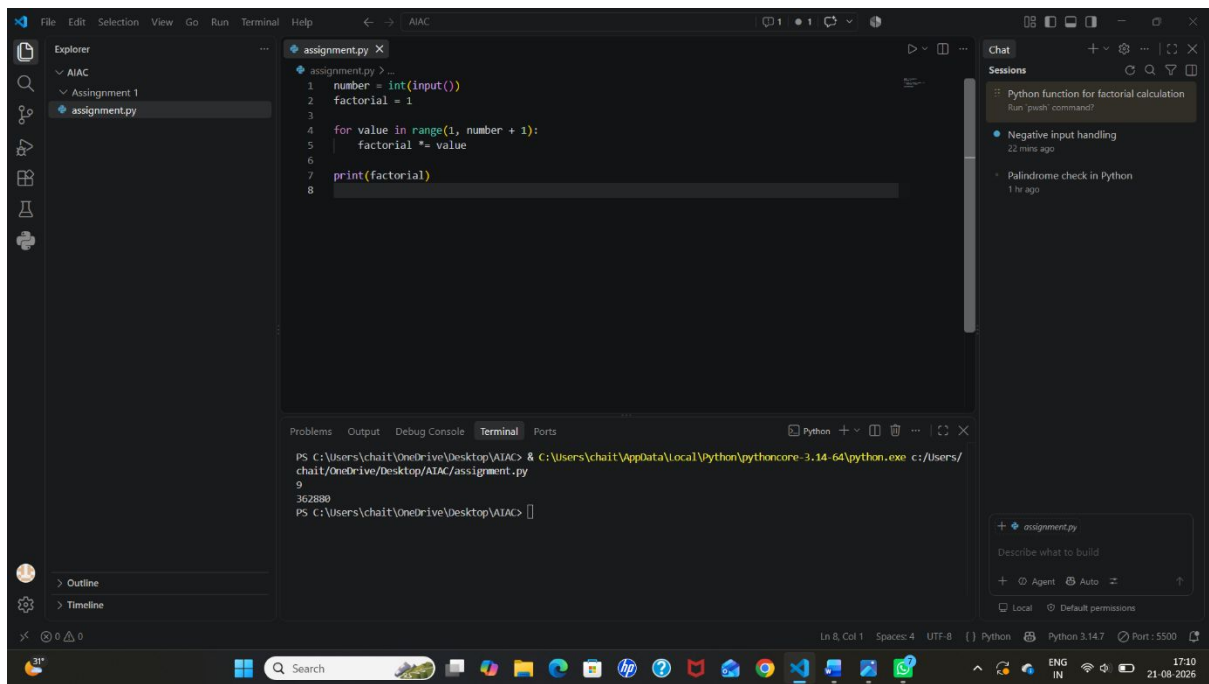
factorial = 1

for value in range(1, number + 1):

    factorial *= value

print(factorial)
```

output:



Q3. Few-Shot Prompting (Armstrong Number Check)

Write a few-shot prompt by providing multiple input-output examples to guide the AI in generating a Python function to check whether a given number is an Armstrong number.

Examples:

- Input: 153 → Output: Armstrong Number
- Input: 370 → Output: Armstrong Number
- Input: 123 → Output: Not an Armstrong Number

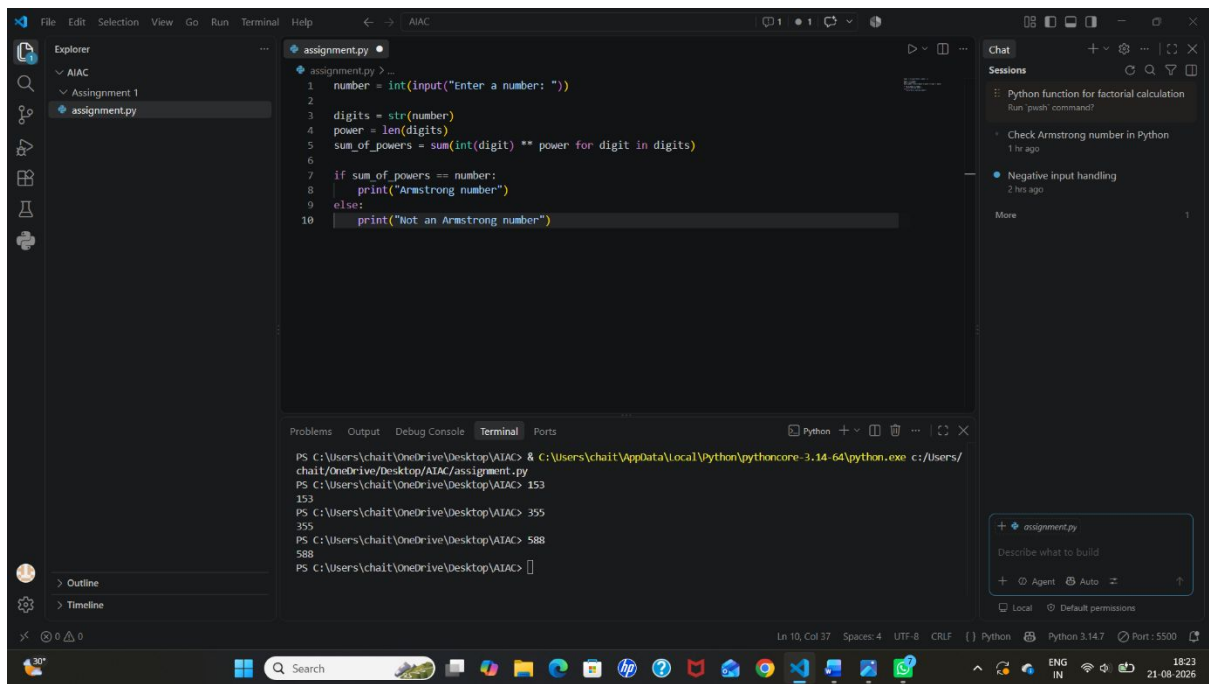
Task:

- Analyze how multiple examples influence code structure and accuracy.
- Test the function with boundary values and invalid inputs.

Prompt: give a python code to check whether a number is Armstrong number or not.

Code:

```
number = int(input("Enter a number: "))
digits = str(number)
power = len(digits)
sum_of_powers = sum(int(digit) ** power for digit in digits)
if sum_of_powers == number:
    print("Armstrong number")
else:
    print("Not an Armstrong number")
```



Q4. Context-Managed Prompting (Optimized Number Classification)

Design a context-managed prompt with clear instructions and constraints to generate an optimized Python program that classifies a number as prime, composite, or neither.

Task:

- Ensure proper input validation.
- Optimize the logic for efficiency.

Compare the output with earlier prompting strategies

Prompt: give a python code to classifies a number as prime, composite or neither

Code:

```

import math

def classify_number(number):

    if number < 2:

        return "neither"

    for divisor in range(2, math.isqrt(number) + 1):

        if number % divisor == 0:

            return "composite"

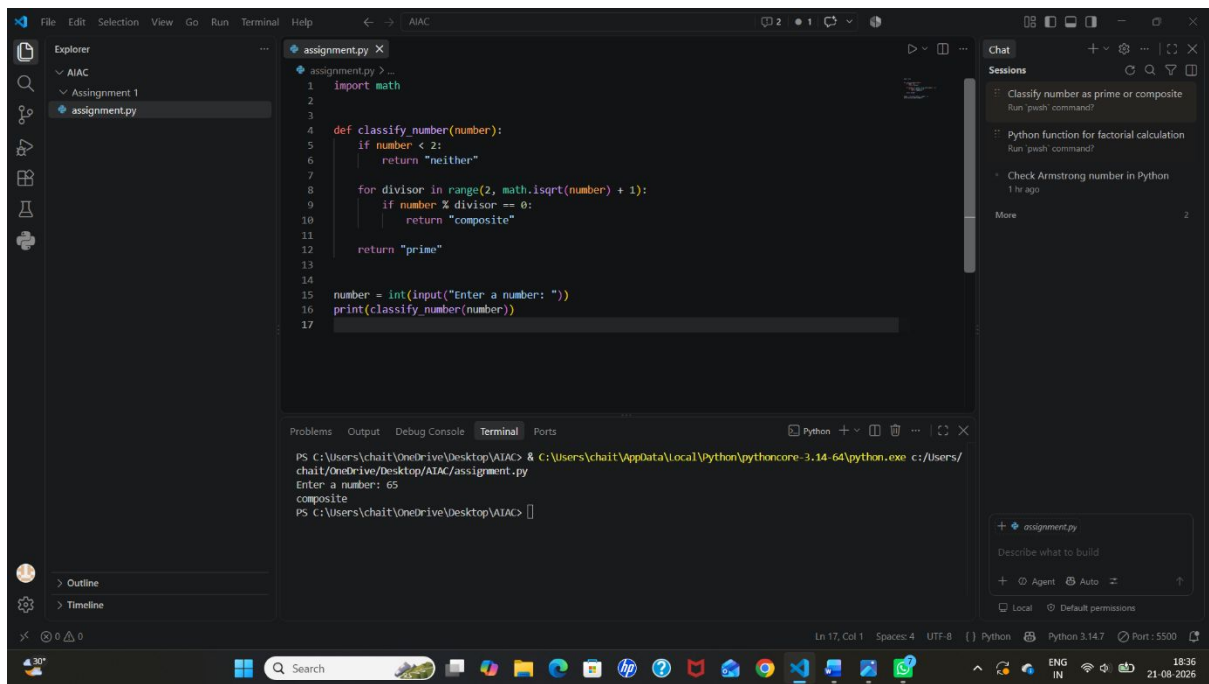
    return "prime"

number = int(input("Enter a number: "))

print(classify_number(number))

```

output:



Q5. Zero-Shot Prompting (Perfect Number Check)

Write a zero-shot prompt (without providing any examples) to generate a Python function that checks whether a given number is a perfect number.

Task:

- Record the AI-generated code.
- Test the program with multiple inputs.
- Identify any missing conditions or inefficiencies in the logic.

Prompt: give a python code to check whether a given number is a perfect number or not.

Code:

```
number = int(input("Enter a number: "))
divisor_sum = sum(
    divisor for divisor in range(1, number)
    if number % divisor == 0
)
if divisor_sum == number:
    print("Perfect number")
else:
    print("Not a perfect number")
```

output:

